

Predicados metalógicos

Curso Programación Lógica 2024

Predicados metalógicos

- Extienden el poder expresivo de Prolog más allá de la Lógica de Primer Orden
- Permiten consultar la instanciación de variables y convertir términos a objetivos.
- Permiten invocar predicados
- **Necesitan conocer el estado de la prueba, y por lo tanto deben conocer la semántica operacional de Prolog**

Predicados de inspección de estructura

- Permiten analizar la estructura de los términos, y crear términos nuevos a partir de otros, o descomponerlos

- Predicados de tipo

 - `integer/1`, `atom/1`, `compound/1`

- Acceso a términos

 - `functor/3`

 - `arg/3`

 - `=.. /2` (univ)

Metapredicados

`atom(+T)` - T es un átomo

`integer(+T)` - T es un entero

`float(+T)` - T es flotante

`number(+T)` - T es entero o flotante

`atomic(+T)` - T es una constante

`var (+T)` - T es una variable **libre**

`compound(+T)` - T es una estructura

Metapredicados

`?- integer(3).`

`?- integer(0.1).`

`?- atom(b).`

`?- atom(f(a)).`

`?- atom(X).`

`?- X=2,atom(X).`

`?- atom(2).`

`?- X=f(b),compound(X).`

`?- X=f(b),atom(X).`

Predicados de inspección

- `var(Term)` - es variable
- `nonvar(Term)` - no es variable libre
- `ground(Term)` - el término está completamente instanciado

?- `nonvar(f(a,X,c)).`

?- `ground(f(a,X,c)).`

Ejercicio 1

Implemente `largo(L,N)` para que funcione con cualquier instancia

Ejercicio 1

Implemente `largo(L,N)` para que funcione con cualquier instanciación

```
% Calculo el largo de una lista  
largo1([_|Xs],N):- largo1(Xs,N1), N is N1+1.  
largo1([],0).
```

¿Qué pasa si el primer argumento no está instanciado?

Ejercicio 1

Implemente `largo(L,N)` para que funcione con cualquier instanciación

```
% Chequea el largo  
largo2([_|Xs],N) :- N>0, N1 is N-1, largo2(Xs,N1).  
largo2([],0).
```

Ejercicio 1

Implemente `largo(L,N)` para que funcione con cualquier instanciación

```
% Chequea el largo  
largo2([_|Xs],N) :- N>0, N1 is N-1, largo2(Xs,N1).  
largo2([],0).
```

¿Qué pasa si el segundo argumento no está instanciado?

Ejercicio 1

Solución

```
% Caso en el que la lista está instanciada, cuento nomás
```

```
largo(Xs,N) :-
```

```
    ground(Xs),
```

```
    largo1(Xs,N).
```

```
% Si la lista es una variable, la creamos instanciada
```

```
largo(Xs,N):-
```

```
    var(Xs),nonvar(N),
```

```
    largo2(Xs,N).
```

Ejercicio 2

Implemente `suma(X,Y,Z)` sobre **enteros**. Tiene que funcionar si hay un argumento no instanciado, y también si solamente Z está instanciado.

Idea:

- Controlar instanciación y dividir los casos
- Si X no está instanciado y los otros sí...
- Ídem Y
- ¿Qué pasa si X y Y no instanciados, y Z sí?

Ejercicio 2

Implemente `suma(X,Y,Z)` sobre **enteros**. Tiene que funcionar si hay un argumento no instanciado, y también si solamente Z está instanciado.

Idea:

- Controlar instanciación y dividir los casos
- Si X no está instanciado y los otros sí...
- Ídem Y
- ¿Qué pasa si X y Y no instanciados, y Z sí? Pista: **between/3**

Ejercicio 2

Implemente `suma(X,Y,Z)` sobre **enteros**. Tiene que funcionar si hay un argumento no instanciado, y también si solamente Z está instanciado.

```
sumaint(X,Y,Z):-  
    var(X),var(Y), ground(Z),  
    % Obtengo X entre 0 y Z  
    between(0,Z,X),  
    % Calculo Y  
    Y is Z - X.
```

Predicados de acceso a términos

- $\text{functor}(\text{?T}, \text{?Nombre}, \text{?Aridad})$ - Verdadero cuando T es un término
Nombre/Aridad
- $\text{arg}(\text{?Arg}, \text{+T}, \text{?Valor})$ - T es un término, Arg es un valor entre 1 y la aridad de T,
Valor se unifica con el Arg-ésimo argumento de T.
- $\text{?T} = \dots \text{?L}$ - L es una lista cuyo primer elemento es el functor de T, y el resto de sus
elementos son los argumentos de T.

Predicados de acceso a términos

?- functor(f(a,b),F,NumArg).

?- functor(f,F,NumArg).

?- arg(N,f(a,b),Arg).

?- Term =.. [f,a,b,c].

?- Term =.. [member,a,[b,c,a]].

?- Term =.. [member,X,[b,c,a]].

?- Term =.. [member,a,[b,c,a]].

?- Term =.. [member,X,[b,c,a]].

?- Term =.. [member,X,[b,c,a]],Term.

?- f(a,b,c) =.. L.

Metavariables

- En Prolog, los datos y los programas se representan igual: a través de términos
- `call(X)` supone que `X` tiene un término con un objetivo, e intenta resolverlo (¡incluyendo variables!)
- Si lo utilizamos en conjunto con `univ`, podemos crear programas y ejecutarlos

Ejercicio 7

`map(+L, +B, ?L2)`: L2 es el resultado de aplicar la función B a todos los elementos de L (B es un predicado binario)

```
?- map([4,3,2,1],doble,L).
```

```
L = [8, 6, 4, 2].
```

Ejercicio 7

`map(+L, +B, ?L2)`: L2 es el resultado de aplicar la función B a todos los elementos de L (B es un predicado binario)

`map([],_,[]).`

`map([X|Xs],B,[X1|Xs1]):-`

`T =.. [B,X,X1], % [doble,4,X1] -> T = doble(4,X1)`

`T, % call(T).`

`map(Xs,B,Xs1).`

Not, revisado.

not utiliza la facilidad de metavariabes

not (o, mejor, fail if)

```
fail_if(Goal):- Goal,!,fail.
```

```
fail_if(Goal).
```

not (o, mejor, fail if)

Regla general: not puede causarnos problemas si los objetivos a negar no están completamente instanciados.

ground/1 puede ser un aliado

Referencias

- The Art of Prolog. Capítulo 10
- Documentación de SWI Prolog